Analyzing the Performance Impact of Page Size on Application Workloads

Uttkarsh Ranjan MT2025128
Department of Computer Science
International Institute of Information Technology, Bengaluru

***Abstract*—Modern operating systems rely on virtual memory and page-based address translation to manage physical memory. A critical design parameter is the page size, which directly influences page fault rates, Translation Lookaside Buffer (TLB) efficiency, and overall memory access latency. This study systematically evaluates the performance impact of page size—comparing 4 KB base pages with 2 MB huge pages—across three representative workloads: a synthetic memory benchmark, a web server (nginx), and a relational database (PostgreSQL). Using Linux Transparent Huge Pages (THP), `perf stat` instrumentation, and controlled experimental methodology, we measure page faults, TLB misses, execution time, throughput, latency, and CPU efficiency. Our results demonstrate that huge pages are *not* universally beneficial: while random-access synthetic workloads achieve a 1.56× speedup with 2 MB pages, both web server and database workloads perform 9–133% *worse* with huge pages due to THP compaction overhead, iTLB pollution, and internal fragmentation. These findings provide workload-aware guidance for page size tuning in production systems.**

***Index Terms*—page size, page faults, TLB misses, huge pages, transparent huge pages, virtual memory, performance evaluation, Linux, nginx, PostgreSQL, synthetic benchmark**

I. Introduction

Modern operating systems rely on virtual memory to efficiently manage physical memory and provide isolation across applications. A key design parameter in virtual memory systems is *page size*, which directly influences page fault behavior, Translation Lookaside Buffer (TLB) efficiency, and memory access latency. While smaller pages reduce internal fragmentation, they increase TLB pressure and page table overhead; conversely, larger pages reduce TLB misses but may negatively impact memory utilization and latency predictability. Linux supports multiple page sizes through base pages (4 KB) and huge pages (2 MB), yet the performance implications of page size selection remain highly workload-dependent.

The objective of this project is to systematically study the relationship between page size, page faults, and end-to-end performance metrics such as latency and throughput. The study focuses on understanding how different memory access patterns interact with page size choices and how these interactions vary across application domains. We evaluate both kernel-level metrics—such as minor and major page faults and TLB miss rates—and application-level performance outcomes to derive workload-aware insights into memory management trade-offs.

A. Project Objectives

The primary objectives of this work are:

- To quantify page fault behavior under different page size configurations.
- To evaluate the impact of page size on TLB efficiency.
- To measure application-level performance changes (latency and throughput).
- To compare behavior across different application domains.
- To identify scenarios where large pages are beneficial or detrimental.

B. Scope

The experimental evaluation covers three workload categories:

1) **Synthetic Memory Benchmark** (C++): controlled sequential, random, and strided memory access patterns over a 1 GB working set.
2) **Web Server Workload**: nginx static file serving with `wrk` load generation, testing small (∼7 KB) and large (50 MB) files under varying concurrency.
3) **Database Workload**: PostgreSQL with sysbench OLTP benchmarks including read-only, read-write, and range scan workloads.

A fourth workload—machine learning inference using PyTorch—is planned but not yet completed.

II. Experimental Setup

A. System Configuration

All experiments were conducted on a fixed hardware and software configuration to ensure reproducibility. Table I summarizes the system specifications.

TABLE I: System Configuration

Parameter	Value
Operating System	Ubuntu 24.04.4 LTS
Kernel	Linux 6.17.0-19-generic (x86_64)
Processor	Intel Core i5-10300H @ 2.50 GHz
Cores / Threads	4 cores / 8 threads
CPU Frequency	0.8 GHz – 4.5 GHz
L1 Cache	128 KB (per core, I+D)
L2 Cache	1 MB (per core)
L3 Cache	8 MB (shared)
Total RAM	16 GB
Available RAM	∼9.7 GB
Swap	4 GB (unused during experiments)
NUMA Nodes	1 (uniform memory access)
Storage	NVMe SSD (∼238 GB), ext4
Default Page Size	4 KB
Huge Page Size	2 MB
THP Mode	`madvise` (controlled per experiment)

B. Page Size Control

Page size configurations are controlled using Linux Transparent Huge Pages (THP). For 4 KB experiments, THP is set to `never`; for 2 MB experiments, THP is set to `always`. In the synthetic workload, finer-grained control is achieved using `madvise()` with `MADV_HUGEPAGE` and `MADV_NOHUGEPAGE` flags.

C. Instrumentation

Performance measurement is performed using:

- **`perf stat`**: hardware and kernel performance counters, including page faults (total, minor, major), data and instruction TLB misses, CPU cycles, and instructions retired.
- **Application-level tools**: `wrk` (web server throughput/latency), `sysbench` (database TPS/QPS/latency).

D. Measurement Methodology

- Each configuration is repeated **3 times** for statistical robustness.
- Both **cold-cache** (page cache dropped, services restarted) and **warm-cache** scenarios are tested.
- No significant background workloads were present during measurements.
- Swap was not utilized, ensuring memory-resident workloads.

III. Metrics Collected

The following categories of metrics are collected across all workloads:

A. Virtual Memory Metrics

Total page faults, minor (soft) page faults, major (hard) page faults, and page faults per unit of work.

B. TLB Metrics

Data TLB (dTLB) load and store misses, instruction TLB (iTLB) load misses, and TLB miss rates.

C. CPU Execution Metrics

CPU cycles, instructions retired, instructions per cycle (IPC), and CPU utilization.

D. Application-Level Metrics

Execution time, throughput (requests/sec or transactions/sec), latency (average, median, p95, p99), and operations per second.

E. Derived Metrics

Percentage reduction in page faults and TLB misses, performance speedup/slowdown, and fault density (faults per unit work).

IV. Synthetic Memory Workload

A. Workload Design

A custom C++ benchmark allocates a 1 GB memory region (128× L3 cache) and accesses it using four patterns:

- **Sequential**: linear traversal of the entire array.
- **Random**: random index access using `rand()` with a fixed seed.
- **Strided-4096**: every 4096th byte (one access per 4 KB page boundary).
- **Strided-65536**: every 65536th byte (one access per 64 KB boundary).

The benchmark is compiled with `g++ 13.3.0` at `-O2` optimization. Page size is controlled via `madvise()` in `madvise` THP mode. Each configuration runs 3 iterations per run with 3 repetitions.

B. Results

Table II presents the summary results across all access patterns and page size configurations.

TABLE II: Synthetic Workload Results Summary

Config	Page Faults	dTLB-Load	dTLB-Store	IPC	Time (s)
seq (4KB)	262,278	797,978	2,541,205	3.56	1.108
seq (2MB)	643	4,550	28,896	3.87	1.120
rand (4KB)	262,276	2,977,257,118	240,865,284	0.21	268.876
rand (2MB)	644	7,981,998	1,465,161	0.32	172.423
stride-4K (4KB)	262,276	560,041	2,762,558	1.24	0.010
stride-4K (2MB)	645	3,456	25,244	0.11	0.009
stride-64K (4KB)	262,276	59,341	2,522,223	1.26	0.002
stride-64K (2MB)	644	2,703	26,251	0.11	0.001

C. Key Findings

1) Page Fault Reduction: Across all access patterns, huge pages reduced page faults from ∼262,276 to ∼644—a ∼**407**× **reduction**. This is consistent with the ratio of page sizes: $1\,\text{GB}/4\,\text{KB} = 262{,}144$ pages vs. $1\,\text{GB}/2\,\text{MB} = 512$ pages. This result is pattern-independent because page faults occur during the pre-fault phase.

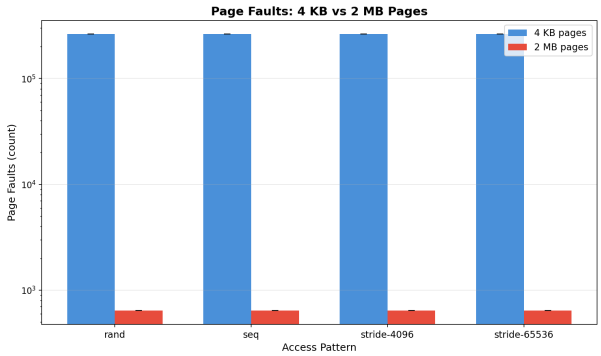

Fig. 1: Page faults: 4 KB vs 2 MB pages (synthetic workload).

2) TLB Miss Reduction: The most dramatic improvements were in dTLB load misses (Table III). Random access with 4 KB pages incurred nearly **3 billion** dTLB load misses; with 2 MB pages, misses dropped by 99.7%.

TABLE III: dTLB Load Miss Reduction (Synthetic)

Pattern	4KB	2MB	Reduction
Sequential	797,978	4,550	99.4%
Random	2,977,257,118	7,981,998	99.7%
Stride-4096	560,041	3,456	99.4%
Stride-65536	59,341	2,703	95.4%

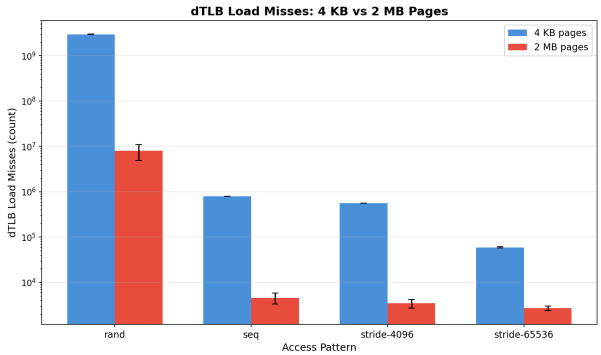

Fig. 2: dTLB load misses: 4 KB vs 2 MB pages (synthetic workload).

Fig. 3: dTLB store misses: 4 KB vs 2 MB pages (synthetic workload).

3) Execution Time: Random access achieved the largest speedup: **1.56**× (268.9 s → 172.4 s), directly correlated with TLB miss reduction. Sequential access showed no meaningful improvement (1.108 s vs. 1.120 s), confirming that high spatial locality makes TLB efficient even with 4 KB pages.

TABLE IV: Execution Time Comparison (Synthetic)

Pattern	4KB (s)	2MB (s)	Speedup
Sequential	1.108	1.120	0.99×
Random	268.876	172.423	**1.56**×
Stride-4096	0.010	0.009	1.06×
Stride-65536	0.002	0.001	2.21×

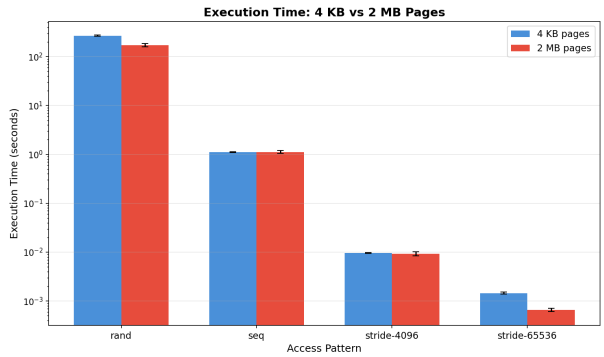

Fig. 4: Execution time comparison (synthetic workload).

4) Instructions Per Cycle (IPC): Random access IPC improved from 0.21 to 0.32 (+52.4%) with huge pages, showing that TLB misses were a major pipeline bottleneck. Sequential access achieved high IPC (>3.5) in both configurations, confirming spatial locality.

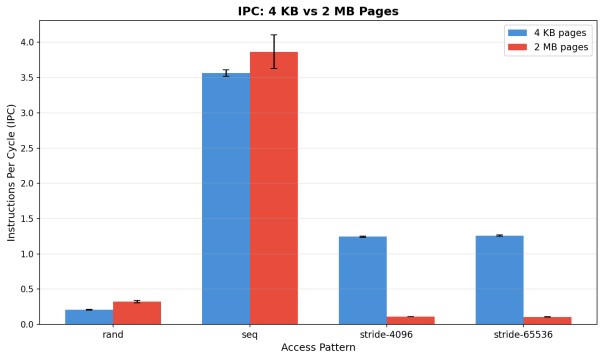

Fig. 5: IPC comparison across access patterns (synthetic workload).

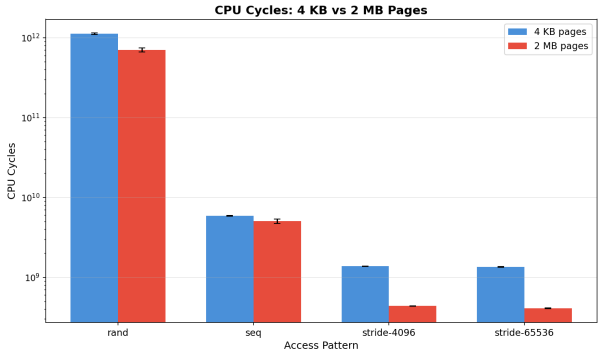

Fig. 6: CPU cycles comparison (synthetic workload).

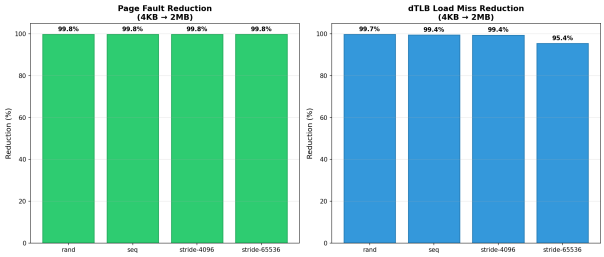

Fig. 7: Percentage improvement summary with 2 MB pages (synthetic workload).

D. Analysis

Why random access benefits most: Each random access potentially touches a different 4 KB page. With ∼1,500 TLB entries and 262,144 pages in the working set, the TLB hit rate is effectively zero. Huge pages reduce the address space to ∼512 pages, making it far more TLB-friendly.

Why sequential access shows no improvement: Sequential access fully utilizes each page before moving to the next. A single TLB miss covers 4,096 (4 KB) or 2,097,152 (2 MB) consecutive byte accesses; in both cases, the per-access TLB miss rate is negligible.

Page faults vs. TLB misses: Page faults are a one-time cost during initial memory population. TLB misses are ongoing during execution. For long-running workloads, *TLB miss reduction* is the dominant performance driver, not page fault reduction.

V. Web Server Workload

A. Workload Design

The web server experiments use **nginx 1.24.0** configured for static file serving with `sendfile on`, `tcp_nopush on`, and logging disabled. Load is generated using **wrk 4.1.0** with 4 threads in `-latency` mode, running locally on port 8088 to eliminate network variability.

Test data: 500 small files (∼6.9 KB HTML each) and 10 large files (50 MB binary each). Page configurations are 4 KB (THP=`never`) vs. 2 MB (THP=`always`). Each run lasts 30 seconds with 3 repetitions, for a total of **48 runs** (2 file sizes × 2 concurrency levels × 2 page configs × 2 cache states × 3 repetitions).

B. Results

TABLE V: Small File Serving Results

Config	Req/s	Avg Lat (μs)	p99 (μs)	PF	dTLB-L	IPC
c10-4KB-cold	115,192	77	519	72	17.0M	0.52
c10-2MB-cold	63,211	157	1,360	77	21.7M	0.44
c10-4KB-warm	107,245	81	504	0	17.4M	0.52
c10-2MB-warm	67,998	143	1,230	0	21.0M	0.45
c200-4KB-cold	150,130	1,353	6,383	156	56.3M	0.49
c200-2MB-cold	92,162	2,110	9,433	153	44.6M	0.42
c200-4KB-warm	145,620	1,353	6,233	2	57.5M	0.49
c200-2MB-warm	79,493	2,390	10,203	3	41.4M	0.41

1) Small File Serving (∼7 KB):

TABLE VI: Large File Serving Results

Config	Req/s	Avg Lat (ms)	p99 (ms)	PF	dTLB-L	IPC
c10-4KB-cold	256	27.2	77.6	76	37.1M	0.85
c10-2MB-cold	120	60.0	145.8	61	11.7M	0.62
c10-4KB-warm	273	32.4	212.9	1	41.8M	0.87
c10-2MB-warm	117	61.2	130.0	9	11.7M	0.62
c200-4KB-cold	180	761.9	1,906.7	560	10.0M	0.74
c200-2MB-cold	135	931.4	1,946.7	558	12.4M	0.62
c200-4KB-warm	161	811.2	1,916.7	63	14.4M	0.69
c200-2MB-warm	140	864.2	1,933.3	159	13.7M	0.62

2) Large File Serving (50 MB):

C. Key Findings

1) 4 KB Pages Consistently Outperform 2 MB Pages: This is the most striking result. Unlike the synthetic workload, 4 KB pages delivered **42–133% higher throughput** than 2 MB pages across all configurations (Table VII).

TABLE VII: Throughput Advantage of 4 KB Pages (Web Server)

Config	4KB	2MB	4KB Adv.
Small, c10, cold	115,192	63,211	+82%
Small, c10, warm	107,245	67,998	+58%
Small, c200, cold	150,130	92,162	+63%
Small, c200, warm	145,620	79,493	+83%
Large, c10, cold	256	120	+113%
Large, c10, warm	273	117	+133%
Large, c200, cold	180	135	+33%
Large, c200, warm	161	140	+15%

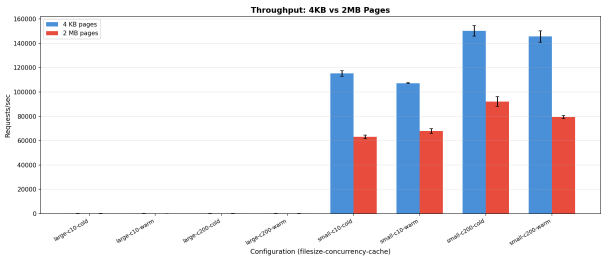

Fig. 8: Request throughput: 4 KB vs 2 MB pages (web server).

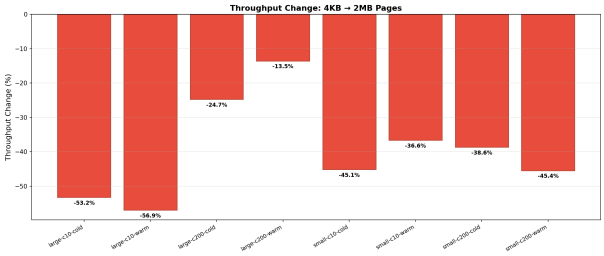

Fig. 9: Percentage throughput change with 2 MB pages (web server).

2) Page Faults Are Not the Bottleneck: Page faults were extremely low (∼60–160 for cold, ∼0 for warm), confirming that nginx and the OS page cache handle demand paging efficiently.

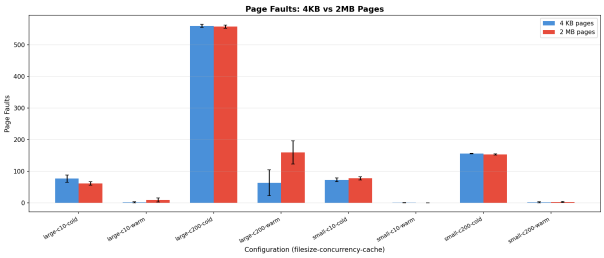

Fig. 10: Page faults comparison (web server).

3) Mixed TLB Behavior: Unlike synthetic workloads, dTLB load misses did *not* consistently decrease with huge pages. For small files, dTLB misses *increased* (17M → 22M at c10). For large files, dTLB misses *decreased* (37M → 12M) but throughput still dropped.

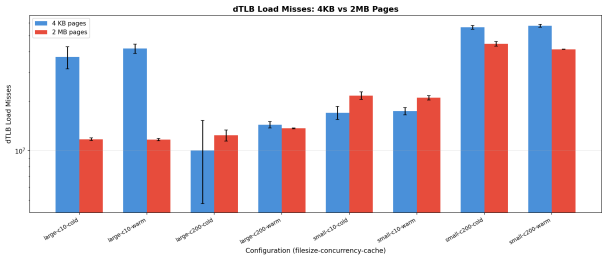

Fig. 11: dTLB load misses (web server).

4) iTLB Pollution with Huge Pages: A critical observation: iTLB load misses were significantly elevated with 2 MB pages for small files. Huge pages for data regions displace instruction TLB entries, causing instruction fetch stalls.

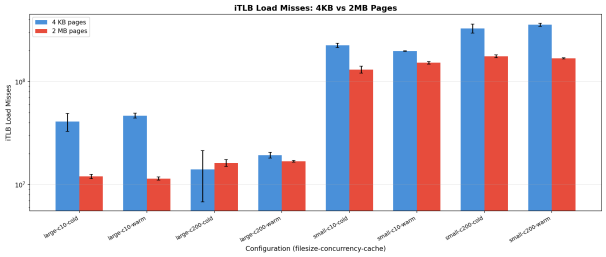

Fig. 12: iTLB load misses (web server).

5) IPC and Latency: IPC was consistently lower with 2 MB pages (–14% to –27%), indicating increased pipeline stalls despite fewer dTLB misses for data. Latency roughly doubled across most configurations.

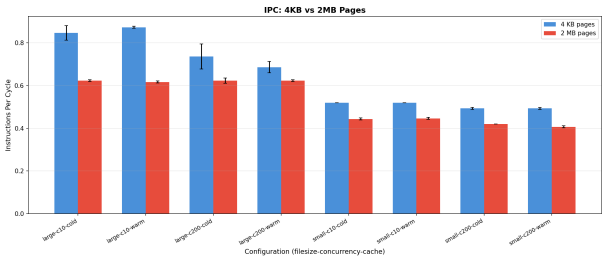

Fig. 13: IPC comparison (web server).

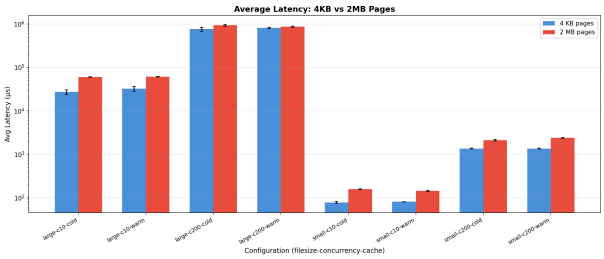

Fig. 14: Average latency comparison (web server).

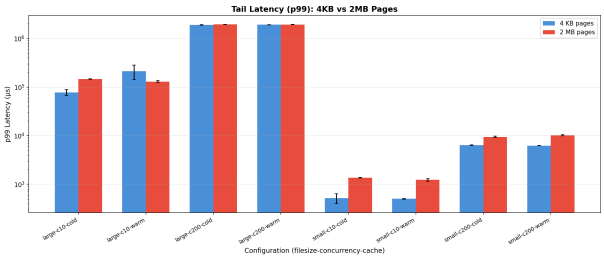

Fig. 15: p99 tail latency comparison (web server).

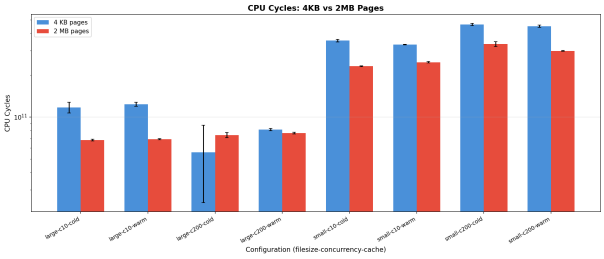

Fig. 16: CPU cycles comparison (web server).

D. Analysis

Several factors explain why 4 KB pages outperform 2 MB pages for nginx:

1) **Memory access pattern**: nginx serves files via `sendfile()`, performing sequential reads through the page cache. Data is copied from page cache to socket buffers via DMA—minimal TLB-dependent user-space memory access occurs.
2) **Internal fragmentation**: small files (∼7 KB) occupy 2 pages at 4 KB but waste most of a 2 MB huge page, reducing effective memory utilization and cache efficiency.
3) **THP overhead**: with THP set to `always`, the kernel's `khugepaged` daemon actively compacts and promotes pages, consuming CPU cycles and causing latency spikes.
4) **iTLB pollution**: huge pages for data regions displace instruction TLB entries. nginx's HTTP handling involves complex branching that is sensitive to iTLB misses.
5) **Page table walk cost**: for nginx's working set (many small files + connection state), the 4 KB page table structure is more cache-friendly.

VI. Database Workload

A. Workload Design

The database experiments use **PostgreSQL 16** with **sysbench 1.0.20** OLTP benchmarks. The dataset consists of 4 tables × 1,000,000 rows (∼1.5 GB). Each run lasts 60 seconds with 4 threads and 3 repetitions, for a total of **36 runs** (3 workloads × 2 page configs × 2 cache states × 3 repetitions).

Workloads tested:

- **Read-only (OLTP)**: 14 SELECT queries per transaction (point lookups + range queries).
- **Read-write (OLTP)**: SELECT, UPDATE, INSERT, DELETE (20 queries per transaction).
- **Range scan**: range queries over contiguous key ranges (100-row ranges).

B. Results

TABLE VIII: Database Workload Results Summary

Config	TPS	QPS	Avg (ms)	p95 (ms)	PF	IPC
RO-4KB-cold	1,745	27,913	2.29	3.31	1,060	0.89
RO-2MB-cold	1,376	22,023	2.90	4.20	1,058	0.90
RO-4KB-warm	1,891	30,259	2.11	2.84	13	1.16
RO-2MB-warm	1,331	21,291	3.01	4.34	12	0.92
RW-4KB-cold	1,175	23,504	3.40	4.41	3,262	0.58
RW-2MB-cold	930	18,601	4.30	5.95	3,317	0.48
RW-4KB-warm	1,000	20,005	4.00	5.51	715	0.47
RW-2MB-warm	916	18,326	4.37	6.15	723	0.47
Range-4KB-cold	7,434	22,301	0.54	0.95	1,965	0.73
Range-2MB-cold	6,743	20,230	0.59	1.05	2,030	0.69
Range-4KB-warm	7,531	22,592	0.53	0.93	13	0.90
Range-2MB-warm	7,502	22,505	0.53	0.93	224	0.81

C. Key Findings

1) 4 KB Pages Outperform 2 MB Pages: 4 KB pages provide 9–42% higher throughput for point lookup workloads, with the gap widening in warm-cache scenarios. Range scans show negligible difference.

TABLE IX: TPS Advantage of 4 KB Pages (Database)

Workload	4KB	2MB	4KB Adv.
Read-only, cold	1,745	1,376	+27%
Read-only, warm	1,891	1,331	+42%
Read-write, cold	1,175	930	+26%
Read-write, warm	1,000	916	+9%
Range, cold	7,434	6,743	+10%
Range, warm	7,531	7,502	+0.4%

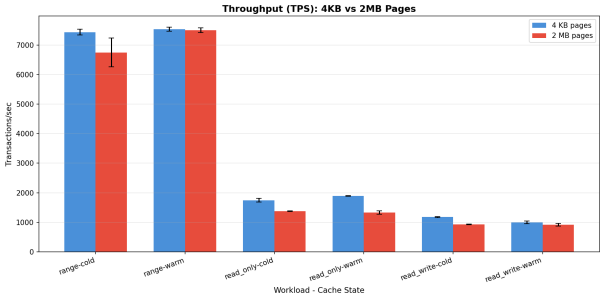

Fig. 17: Transactions per second: 4 KB vs 2 MB pages (database).

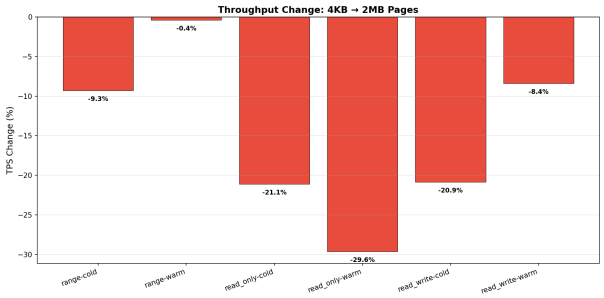

Fig. 18: TPS percentage change with 2 MB pages (database).

2) Page Faults Are Similar Between Configurations: Page faults are low (∼1K–3K cold, ∼10–700 warm) and nearly identical between configurations. PostgreSQL's internal buffer pool mediates memory access, neutralizing page fault differences.

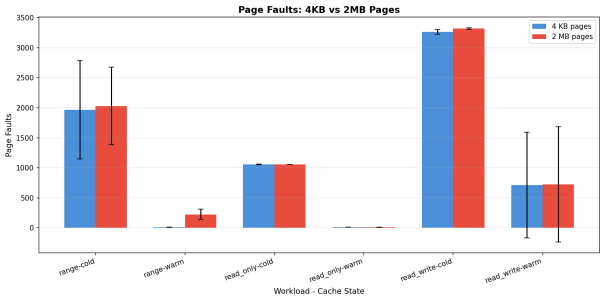

Fig. 19: Page faults comparison (database).

3) Latency: The p95 tail latency increase with 2 MB pages is notable for read-only workloads (+27% cold, +53% warm), indicating THP background compaction causes latency spikes.

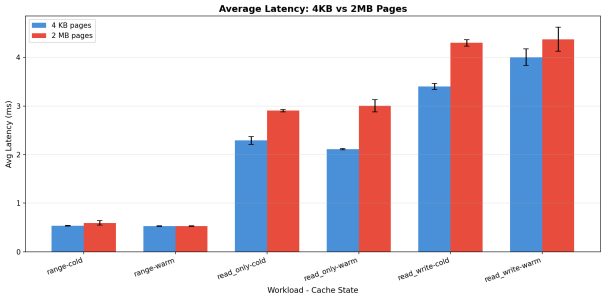

Fig. 20: Average latency comparison (database).

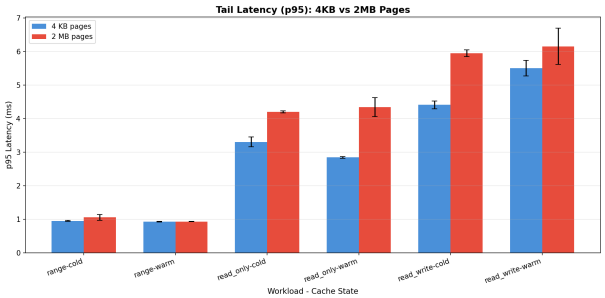

Fig. 21: p95 tail latency comparison (database).

4) IPC and TLB Metrics: In the warm read-only scenario, 4 KB pages achieve markedly higher IPC (1.16 vs. 0.92), suggesting huge pages increase pipeline stalls during point lookups.

Fig. 22: IPC comparison (database).

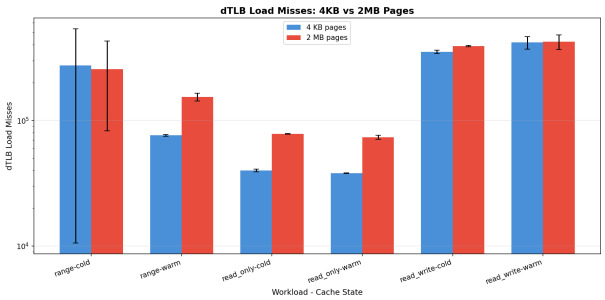

Fig. 23: dTLB load misses (database).

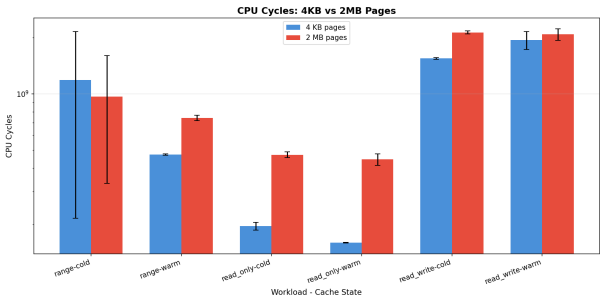

Fig. 24: CPU cycles comparison (database).

D. Analysis

1) **PostgreSQL manages its own buffer pool**: `shared_buffers` provides an application-level cache on top of the OS page cache. Most data access hits the buffer pool, bypassing OS page faults entirely.
2) **B-tree index traversal**: point lookups traverse non-contiguous pages. With 2 MB pages, the additional memory loaded per TLB entry is mostly wasted.
3) **THP compaction overhead**: the `khugepaged` daemon competes with PostgreSQL's I/O-intensive workload.
4) **Memory efficiency**: PostgreSQL's buffer pool uses 8 KB pages internally. Each 2 MB OS page holds 256 PostgreSQL pages, but only a few may be actively accessed.

Range scans show minimal difference because contiguous key ranges produce sequential B-tree leaf page access—high spatial locality makes page size irrelevant, consistent with the synthetic sequential result.

VII. Cross-Workload Comparison

Table X summarizes the page size impact across all three workloads, demonstrating that the benefit of huge pages is highly workload-dependent.

TABLE X: Cross-Workload Page Size Impact

Aspect	Synthetic (Random)	Web Server	Database
Page size winner	**2 MB** (+56%)	**4 KB** (+33–133%)	**4 KB** (+9–42%)
Access pattern	Truly random, user-space	Kernel (sendfile)	Index traversal
Page faults (4KB)	High (262K)	Very low (∼65)	Low (∼1K–3K)
TLB miss reduction	99.7% with 2MB	Mixed / worse	No clear benefit
Why 4KB wins	N/A	THP overhead, iTLB	THP overhead, buffer pool

Key insights from the cross-workload comparison:

1) **Only purely random, user-space memory access benefits from huge pages**. The synthetic random workload—where each access touches a different page—is the only scenario with clear speedup.
2) **Kernel-mediated access (sendfile) bypasses TLB benefits**. nginx's use of `sendfile()` means data never passes through user-space TLB entries, negating the advantage of huge pages.
3) **Application-level buffering masks page faults**. PostgreSQL's buffer pool absorbs most memory accesses before they reach the OS, making page size nearly irrelevant for page fault behavior.
4) **THP `always` mode is harmful for server workloads**. The background compaction by `khugepaged` consistently degrades performance for both nginx and PostgreSQL.
5) **`madvise` mode is the correct default**. It allows applications to opt-in to huge pages only where beneficial.

VIII. Conclusions

This study provides a comprehensive, workload-aware characterization of how page size influences page fault behavior and application performance. Our key conclusions are:

1) **Huge pages (2 MB) universally reduce page faults and TLB misses**, but the performance impact depends entirely on the memory access pattern. Page faults decreased by $\sim$407$\times$ across all workloads with huge pages.
2) **Random user-space access benefits most**: the synthetic random workload achieved a 1.56$\times$ speedup, with dTLB miss reduction of 99.7%. This result is directly applicable to workloads like hash tables and graph algorithms.
3) **Web server performance degrades with huge pages**: nginx throughput dropped 33–133% with 2 MB pages due to THP compaction overhead, iTLB pollution, and internal fragmentation from small files.
4) **Database performance degrades with huge pages**: PostgreSQL OLTP throughput dropped 9–42% with 2 MB pages because the internal buffer pool masks page fault benefits while THP overhead increases tail latency.
5) **TLB miss reduction is the primary performance driver**, not page fault reduction. Page faults are a fixed one-time cost; TLB misses are ongoing during execution.
6) **THP `madvise` mode is the recommended configuration** for production systems, allowing workload-specific opt-in rather than system-wide huge page promotion.
7) **IPC is a useful diagnostic indicator**: low IPC ($<$0.5) signals TLB/memory bottlenecks where huge pages may help, while high IPC ($>$1.0) indicates spatial locality that already minimizes TLB overhead.

These findings demonstrate that page size tuning must be informed by workload memory access patterns. Large pages are not universally beneficial, and indiscriminate

use of THP `always` mode can significantly harm performance for server workloads.

IX. Future Work

The following extensions are planned:

- **Machine Learning Workloads**: evaluate PyTorch model loading and inference under different page sizes to characterize the impact on contiguous memory allocation patterns.
- **1 GB Huge Pages**: investigate the effect of even larger page sizes on workloads that allocate very large memory regions.